

Отчёт по индивидуальному заданию №1

Архитектура компьютера
и операционные системы

Вариант 24 «Танковые бои»

Выполнил:
Махмутов Марсель Наилевич
Группа: 252

2026

Условие и устройство программы

Вариант 24, «Танковые бои». На клеточном поле действуют танки двух сторон: они наблюдают цели, перемещаются и стреляют. На каждом такте стороны независимо выбирают действия по одному состоянию поля, затем выполняют их по общим фазам. Порядок обхода участников не должен давать преимущества. Требуется консольная модель с выводом событий и итоговой статистики. Завершение: уничтожение одной или обеих сторон, предел тактов либо прерывание пользователем.

Программа написана на С для Linux с использованием `libc` и системных вызовов. Работает один процесс и один поток: общий цикл последовательно имитирует действия участников. В коде есть комментарии к данным и основным действиям.

Сущность	Представление и поведение
Танк	Tank: сторона, номер, координаты, прочность, дальность. Выбирает движение и цель.
Координатор стороны	Team: стратегия и статистика. <code>team_plan</code> формирует намерения танков стороны.
Снаряд	Shot: стрелявший танк, клетка цели, точность и такт прилёта.
Поле и состояние боя	Battle: карта, массивы танков и снарядов, стороны и текущий такт.

В `main.c` находятся запуск, общий цикл и завершение; в `config.c` чтение и проверка данных; в `battle.c` правила боя; в `output.c` карта и итоги. Объявления вынесены в соответствующие заголовочные файлы. Используются массивы: поле до 40×20 , до 32 танков, полёт снаряда до 20 тактов. Запас 672 снаряда учитывает новые выстрелы до обработки прилетевших.

Правила и порядок такта

Координаты начинаются с нуля. Стратегия `hold` сохраняет позицию, `advance` приближает танк к ближайшему противнику до дальности стрельбы. Расстояние равно сумме разностей координат по модулю. За такт возможен переход в соседнюю клетку вверх, вправо, вниз или влево; при равенстве выбирается первое направление. Занятые в начале такта клетки и препятствия недоступны. Если несколько танков выбрали одну клетку, все остаются на месте.

Танк стреляет не чаще одного раза за такт по ближайшей цели в пределах дальности, при равенстве выбирает меньший номер. Стрельба навесная: препятствия не закрывают цели. Снаряд летит в выбранную клетку; при `flight=1` выстрел первого такта прилетает на втором. Вероятность точного выстрела задаётся в процентах. Попадание снимает 1 единицу прочности у танка в клетке, в том числе союзного. Пустая клетка и неточный выстрел не дают урона. Уничтоженная машина остаётся препятствием.

1. Обе стороны наблюдают поле, выбирают движение и цели; поле ещё не меняется.
2. Регистрируются выстрелы из начальных позиций, разрешаются конфликты движения, танки перемещаются.
3. Обрабатываются прилетевшие снаряды. Урон сначала накапливается, затем одновременно применяется ко всем танкам.
4. Проверяется корректность состояния, выводится поле и определяется завершение.

Так сохраняются независимость планирования, однозначность конфликтов и запрет двух действующих танков в одной клетке. Прочность не опускается ниже нуля. Уничтоженные танки больше не действуют, но их выпущенные снаряды продолжают полёт до конца боя. Порядок фаз, тип стрельбы и правила обломков закреплены в программе.

Интерфейс, запуск и завершение

Сборка и запуск из каталога проекта:

```
cmake -S . -B build
cmake --build build
./build/tanks
./build/tanks --config data/demo.txt --seed 24 --delay 200 --log battle.log
./build/tanks --config data/conflict.txt --limit 0
./build/tanks --help
```

Без параметров используется поле 12×8 с двумя танками каждой стороны. --config выбирает сценарий; --seed задаёт начальное значение генератора случайных чисел; --delay задержку между тактами в миллисекундах; --limit предел тактов, 0 означает отсутствие предела; --log файл итогов. Аргументы имеют приоритет над файлом. Значения по умолчанию: seed=24, delay=200, limit=100, точность 80%, полёт 1 такт, обе стороны используют advance.

Пример файла данных:

```
field 5 2
limit 20
accuracy 100
flight 1
seed 24
delay 200
team A advance
team B hold
map
.....
..#..
tank A 1 0 0 3 4
tank B 1 4 0 3 4
```

Обязательны field и map. После map идут строки поля, затем записи tank <A|B> <номер> <x> <y> <прочность> <дальность>. Номера 1..99 уникальны внутри стороны, прочность 1..100, дальность 1..60. Точность 0..100, полёт 1..20, задержка 0..10000, предел 0..1000000. Вне карты разрешены пустые строки и комментарии после #. Проверяются границы, числа, символы карты и размещение танков. Ошибки выводятся с пояснением. При ошибках разбора сценария указывается строка.

На экране отображаются наблюдение, намерения, споры за клетки, движение, выбор цели, выстрелы, попадания, урон, уничтожение и поле после такта. Символы: . свободная клетка, # препятствие, A/B танки, X обломки. Под картой указаны номера, координаты и прочность танков.

Чтение использует open, read, close, вывод выполняется через dprintf по дескрипторам. В battle.log либо файл из --log записываются причина завершения, число тактов, оставшиеся танки, потери, выстрелы, попадания, выстрелы без урона и снаряды в полёте. Журнал перезаписывается; совпадение с входным файлом проверяется до его очистки. Полный вывод можно сохранить через > events.txt.

Обычное завершение наступает при победе, одновременном уничтожении сторон или достижении предела. Если на предельном такте уничтожена сторона, фиксируется победа или ничья. Если обе стороны ещё имеют живые танки, бой заканчивается по пределу без победителя. Пустое поле сразу даёт ничью. В режиме --limit 0 возможен бесконечный бой, например спор за клетку в conflict.txt. Ctrl+C (SIGINT) и SIGTERM прерывают и конечный, и бесконечный режимы. Обработчик только устанавливает флаг; основной код сохраняет итоги и закрывает журнал. При сигнале во время такта он завершается, задержка прерывается. Коды выхода: 0 при обычном завершении, 130 для SIGINT, 143 для SIGTERM; ошибки дают 1. Оставшиеся снаряды после конца боя не обрабатываются.

Проверки и оценка реализации

Проверки выполнены на Linux с GCC 11.4.0. Результаты в таблице получены с seed=24 и задержкой 0. Команда: `bash tests/run_tests.sh ./build/tanks`. Результат: OK: 59 проверок пройдено. Данные находятся в data, скрипт в tests/run_tests.sh.

Сценарий	Полученный результат	Тактов
demo.txt	Победа В; А: 0 живых, В: 1	7
win_a.txt, win_b.txt	Победа соответствующей стороны	2
draw.txt, multi_hit.txt	Оба уничтожены / два попадания в В, победа А	2
conflict.txt, miss.txt	Предел: спор за клетку / точность 0	3
blocked.txt, wreck.txt	Предел: препятствие / обломки мешают движению	4
escape.txt	Предел: цель уходит из клеток обстрела	4
empty.txt	Ничья без танков	0
bad_map.txt, bad_overlap.txt	Ошибка длины строки / два танка в клетке; код 1	Не начат

В demo.txt каждая сторона сделала 8 выстрелов: А нанесла 5 попаданий, В 6; без урона 1 и 0, в полёте осталось по 2 снаряда. В draw.txt оба танка с прочностью 1 уничтожены на втором такте: проверяется одновременное применение урона. В conflict.txt --limit 0 --delay 1000 SIGINT после первого такта дал код 130 и журнал с причиной «Прервано пользователем», по одному живому танку, без выстрелов и потерь.

Скрипт также проверяет повторяемость при одном seed, перестановку записей танков, ошибочный ввод, защиту исходного файла от перезаписи журналом, SIGINT и SIGTERM в обоих режимах. При 32 танках и полёте 20 после 50 тактов остаётся 640 снарядов, массив не переполняется.

Ограничения: движение не ищет обход препятствий; `rand()` может давать другие результаты в другой libc; ошибки вывода событий во время боя не проверяются, поэтому при закрытии получателя программа может продолжить работу без видимого вывода. Для застрявших танков доступны предел тактов и прерывание. При доработке собственный вывод заменён на `dprintf`, разбор строк выполняется по позиции в буфере и через `strtok`; проверки сигналов ожидают первый такт и охватывают конечный и бесконечный режимы.

Использование ИИ

Первый промпт: «Напиши код по следующему устройству программы. Использую C, один общий цикл без дополнительных потоков. В main.c будут запуск и завершение, в config.c чтение исходных данных, в battle.c расчёт боя, в output.c карта и запись итогов. Танки и снаряды хранятся в массивах структур. Сначала обе стороны выбирают действия по одному состоянию поля, затем выполняются выстрелы, решаются споры за клетки, применяются перемещения и одновременно учитывается урон. Для файлов используй дескрипторы, для вывода `dprintf`. При Ctrl+C нужно сохранить итоги и закрыть файл. Перед написанием кода ознакомься с условием задания, предложи изменения по решению, согласуй со мной, только после этого приступай к написанию кода».

В дальнейших запросах я уточнял правила движения и стрельбы: стратегии hold и advance, разрешение споров за клетки, задержку попадания и сохранение обломков. Также просил реализовать чтение сценария, проверку входных данных, параметры командной строки и запись итоговой статистики. Затем отдельно рассматривал применение урона, обработку сигналов и автоматизацию проверок.

1. «Проверь обработку попаданий в battle.c: урон сначала накапливается в `dmg`, затем применяется ко всем танкам. Разбери пример взаимного уничтожения двух танков с прочностью 1 и проверь, зависит ли результат от порядка обхода».
2. «В обработчике сигнала я хочу только устанавливать флаг `volatile sig_atomic_t`, а запись итогов и закрытие файла выполнять в основном коде. Проверь остановку через SIGINT и SIGTERM во время задержки и выполнения такта».
3. «Автоматизируй проверку сценариев из data: победа каждой стороны, ничья, конфликт перемещений, ошибочные данные и прерывание бесконечного боя. Проверяй код завершения и содержимое журнала».